Setting Up a Small Research Team in Hermes Bot Mode



A practical, step-by-step guide to building and running a small multi-agent research team inside Hermes Agent. You define one bot per role, keep the team lean, choose tools deliberately, and review each handoff as a human before anything reaches your knowledge base.

This guide adapts the method shared by the Wanderloots Hermes Bot Mode tutorial and its quick-reference notes. Credits and links are at the end. The wording here is written fresh as a standalone how-to, so you can follow it without watching the video or holding the source open, but the source notes are worth reading alongside it.

© Billy Forrest · training.forrest.one

© Billy Forrest @ https://training.forrest.one

What you will build

A three-role research team that works under your review:

- **Orchestrator** clarifies the outcome, delegates the research, reviews the returned report for relevance, citations, uncertainty and gaps, and asks you before passing accepted work onwards.
- **Researcher** investigates bounded questions against current and original sources and returns a concise, cited report.
- **Librarian** (optional) proposes and organises knowledge you have approved.

If you only want a reviewed report, you can stop after the Researcher. The Librarian only matters once you want approved knowledge written into a wiki.

By the end of the guide you will have tested the communication between the bots with a harmless message, run one full research thread under your review, and kept the two approvals (research, then wiki) separate.

Before you start

You need a running Hermes installation with Bot Mode available and a working model connection for each bot. Nothing else is a hard prerequisite.

Confirm you can start the Bot Mode interface and create a bot. If you have never opened it, the entry point is the Bots area (Bots → New Bot in the guide below), so open it once to confirm the page loads.

Decide how much of the workflow you want. A two-bot team (Orchestrator and Researcher) is enough to produce a reviewed report. Add the Librarian only when you are ready to manage approved knowledge.

Prepare a working folder that both the Orchestrator and the Researcher can reach, because the Researcher saves its report to disk and the Orchestrator reads it back. More on why this matters later.

Likely errors and fixes: - The Bots page is empty or the create option is missing. Fix: update Hermes and confirm Bot Mode is installed and enabled before continuing. - No model is configured. Fix: connect and test a working model for the bots first, then return here.

Step 1: Give each bot a clear job

The single most important step is a precise, written job description for each bot. A vague SOUL produces vague behaviour.

Step 1.1: Create a bot

In the Bots area (Bots → New Bot), choose a name, a title and a description for each role.

Leave the SOUL field blank the first time you create a role and let the assistant generate a draft from your name, title and description. Then open the result, read it, and edit it to suit your own needs. The descriptions below are a starting point, not finished exports, and you will almost always want to tune the wording for your context.

© Billy Forrest @ https://training.forrest.one

Step 1.2: Write the Orchestrator job

> *Clarify the outcome and acceptance criteria. Delegate research to the Researcher. Review the returned report for relevance, citations, uncertainty and gaps. Request at most one focused revision. Ask me before handing accepted work to the Librarian. Do not repeat the Researcher's work or write wiki content yourself.*

What the role is for: the Orchestrator keeps the team on track and is the bot you actually talk to. It delegates rather than duplicating the research, and it never writes knowledge content itself.

Step 1.3: Write the Researcher job

> *Investigate bounded questions using current and original sources. Save a concise, cited Markdown report in [RESEARCH FOLDER]. Include uncertainty and counterevidence. Return the report and file path to the Orchestrator, not directly to the Librarian. Do not write to the wiki.*

What the role is for: the Researcher does the investigation and returns a traceable report. Note the constraints that keep the workflow clean: the report has a saved location, it goes back to the Orchestrator, and the Researcher never touches the wiki.

Step 1.4: Write the Librarian job (only if you want the wiki step)

The Librarian is the role that proposes and organises approved knowledge. In the source material this was built on the llm-wiki and llm-wiki-review skills, which handle the review of proposed content against its sources. Give the Librarian a job description that says it must run a review of any proposed knowledge and hold changes until you approve them.

Likely errors and fixes: - A bot behaves like a general-purpose assistant instead of its role. Fix: the SOUL is too vague or was left at the generated default. Tighten it to name the role, its single responsibility, and what it must not do. - Two bots duplicate the same work. Fix: your job descriptions overlap. Make ownership explicit, as in "the Researcher investigates; the Orchestrator reviews; it does not repeat the research".

Step 1.5: Reload and check the change

After you edit a bot's SOUL, reload or restart its profile as your Hermes version requires. Then ask the bot:

> *What changed about your responsibilities and handoffs?*

What this does: confirms the bot read its edited job description correctly before you trust it with real work. If the bot's answer does not match the job you wrote, fix the SOUL before continuing.

Step 2: Choose the tools and skills deliberately

Three layers do different jobs: the SOUL defines the role, skills provide the procedures, and tools provide executable powers. The common mistake is giving every bot everything. Narrow the toolset first, and add powers only when a task actually needs them.

Installed skills are progressively loaded, so one bot inheriting another's skill library is not the same as loading every procedure into every request. Only enable the procedures a role truly uses.

Step 2.1: Orchestrator tools

Keep: file operations, skills, clarifying questions, and memory.

Make optional: terminal and processes for bounded checks such as a word count, session search for prior work, and code execution for a specific calculation or transformation.

© Billy Forrest @ https://training.forrest.one

No dedicated companion skill is required for the Orchestrator if its SOUL carries the coordination instructions, as written in Step 1.

Step 2.2: Researcher tools

Keep: web search and scraping, file operations, terminal and processes, skills, and clarifying questions.

Skills: the grounded-citations skill, so the report cites verifiable sources.

Make optional: memory only for operating preferences, browser automation for sources that ordinary web tools cannot inspect, vision for figures and scans, code execution for a defined analysis, and image generation only for a visual you specifically request.

Step 2.3: Librarian tools

Keep: file operations, terminal and processes, skills, and clarifying questions.

Skills: llm-wiki and llm-wiki-review, and a helper for mounted paths if your wiki runs under Docker.

Make optional: memory for stable preferences, and session search for a specific prior decision.

Step 2.4: Leave the heavy machinery off

For this beginner workflow, leave task delegation, computer use, agentised automation (A2A), unrelated services and unneeded generation tools disabled. Kanban, cron, todo, and text to speech are not prerequisites.

Two clarifications that commonly confuse newcomers:

- A bot @mention does not need task delegation enabled. Task delegation creates temporary subagents; messaging your persistent teammates is native Bot Mode communication and is always available.
- Keep independent web research disabled for the Orchestrator and the Librarian. Only the Researcher should be searching the web; otherwise two agents chase the same question with no single owner.

Likely errors and fixes: - Bots behave slowly or spend turns on irrelevant tools. Fix: narrow the enabled tools to the keep-list for the role, then re-test. - The Researcher's report has no usable citations. Fix: the grounded- citations skill is missing or disabled for the Researcher.

Step 3: Check the settings that matter most

Four settings make or break a small team.

Warm backends and idle timeout. More warm backends let the team respond faster but use more RAM. A recorded configuration used five warm backends and a ten minute backend idle timeout. Adjust to your machine and how many bots are active at once.

Models. Start with a working model connection for every bot. If you plan to use cheaper or local models for specialist roles, test them on the same task before assuming they match the quality of your main model. Do not assume savings without measuring quality.

Memory. A memory system is optional for this workflow. When you do use one, store operating preferences (such as how the user likes reports), not research claims or citations, which belong in the written report itself.

© Billy Forrest @ https://training.forrest.one

Shared files. Both the Orchestrator and the Researcher must be able to reach the report file. Test this with a harmless file before you start any real research, so you are not three steps into an investigation before you discover the bots cannot see each other's work.

> *Workspace means starting directory, not a filesystem sandbox. Docker mounts and permissions determine what a bot can actually reach, and local files read by a cloud model may leave your computer. Use a test folder or a supported, backed up vault while you learn, and understand the safety implications before you give a bot access to sensitive local data.*

Likely errors and fixes: - The Researcher says it saved a report but the Orchestrator cannot open the path. Fix: the two bots use different working directories or permissions. Use a shared folder you tested in advance (see Step 'Before you start'). - Responses are slow and the machine is struggling. Fix: reduce the number of warm backends.

Step 4: Test communication, then run a research thread

Never start with a real question. Test that the bots can talk to each other and that attribution works first.

Step 4.1: Test the team talks

Create a fresh room with the bots you are going to use and send a harmless message:

> *Everyone, say hi and give me your favourite colour. Then @orchestrator, tell me each bot's favourite colour.*

What to look for: the team replies, answers are attributed to the right bot, and the conversation settles rather than looping.

Likely errors and fixes: - Bots keep talking past each other or never settle. Fix: the roles or the room are misconfigured. Check each bot has the right teammates and the orchestration handoff is clear before proceeding. - One bot replies for itself and another. Fix: attribution is working if each answer names its author; if not, check the roster handles.

Step 4.2: Run a bounded research thread

Once the team talks, start a research thread with tight constraints.

> *@orchestrator, research current best practices for multi-agent workflows, focusing on evidence relevant to 2026. Use at most ten sources and a report under 500 words with traceable citations. Delegate to the Researcher, review the result, and request at most one focused revision. Ask me before involving the Librarian.*

What this does and why the limits matter: naming the source cap, the word cap and the single-revision cap keeps the thread bounded and reviewable. Without limits, a research thread can sprawl across many sources and revisions.

Step 4.3: If the relay needs a nudge

If the automatic relay reaches its per-send cap, the room remains usable. Inspect the pending request and send another message to the right bot:

> *@researcher, please continue with the focused revision requested by the Orchestrator.*

An important distinction: nudging the Researcher to continue the revision is not the same as approving the research. Approval is a separate, explicit act (Step 5).

Likely errors and fixes: - Nothing happens after the @orchestrator message. Fix: check the bot is in the room and its mention handle is correct. A bot only acts when it is both present and mentioned. - The report ignores the word or source cap. Fix: tighten the constraints in the

© Billy Forrest @ https://training.forrest.one

Orchestrator's SOUL or restate them in the message. Bots follow a clearly bounded instruction better than an open-ended one.

Step 5: Keep the two approvals separate

The workflow has two independent approval points. Keeping them separate is what lets you trust the final output.

First approval: the report. Once you have read the Researcher's report and it is acceptable, hand the content towards the knowledge stage:

> *@orchestrator, I approve the handoff. Move this report to [RAW INBOX PATH] and ask [LIBRARIAN HANDLE] to run llm-wiki-review. Hold wiki changes for my approval.*

Second approval: the wiki content. The Librarian proposes the knowledge entry and runs its review, but does not publish anything until you say so. Inspect the proposed content and its sources, then approve, revise, reject, or defer it.

After you approve, verify the actual wiki note and its links yourself. Do not trust a graph animation or a status message alone; skill-based review is instructional, it is not deterministic enforcement.

Likely errors and fixes: - Content appears in the wiki before you approved it. Fix: the Librarian's SOUL does not require approval before writing, or a bot wrote directly to the wiki. Reinstate the "hold changes for my approval" rule. - The verified note does not match what the Librarian described. Fix: check the published note and links directly, then correct the Librarian's process so it reports reality rather than intent.

Step 6: Save operating preferences, not findings

Close out a session by teaching the team how you like to work. Ask a relevant bot:

> *Save useful operating preferences from this session to built-in memory, not research claims. Tell me what changed.*

What this does: stores how you prefer the team to operate (format, length, citation style, revision discipline) without cluttering memory with statements that belong in the research record.

Likely errors and fixes: - The bot saves research claims instead of preferences. Fix: your memory policy is not clear. Memory is for how the user works; the report file is for what was found.

Recap and what to try next

You now have a small, human-reviewed research team:

- one bot per role, each with a tight written job description;
- a deliberately narrow toolset per role;
- a tested communication path before real work;
- bounded research threads under your review; and
- two separate approval gates before anything reaches your knowledge base.

The takeaway from the source material is worth repeating: keep the team small, and improve its instructions, capability choices and handoffs through repeated tests before you add more bots. For a quick question a single capable agent is often still the better choice.

© Billy Forrest @ https://training.forrest.one

If you want to go further, the natural next step is the deterministic librarian workflow that reviews proposed knowledge against its sources, which the source notes cover in a separate article (the "trusted LLM wiki" kit). Treat this guide as the on-ramp to that.

Quick reference

Roles to build:

Role Owns Must not do Orchestrator	clarifying scope, delegating, reviewing	duplicate research or write wiki content
Researcher	bounded investigation, cited report	write to the wiki
Librarian	proposing and organising approved knowledge	publish without approval

First test, then research, then approve:

1. Test the team with a harmless favourite-colour message, check attribution.
2. Run a bounded research thread with source and word caps and one revision.
3. Approve the report, then approve the wiki change, separately.
4. Verify the final note and its links yourself.

Useful prompts to copy:

> *Everyone, say hi and give me your favourite colour. Then @orchestrator, tell me each bot's favourite colour.*
>
> *@orchestrator, research current best practices for multi-agent workflows, focusing on evidence relevant to 2026. Use at most ten sources and a report under 500 words with traceable citations. Delegate to the Researcher, review the result, and request at most one focused revision. Ask me before involving the Librarian.*
>
> *@orchestrator, I approve the handoff. Move this report to [RAW INBOX PATH] and ask [LIBRARIAN HANDLE] to run llm-wiki-review. Hold wiki changes for my approval.*

Credits and sources

This guide is an original, adapted how-to based on the method and prompts published by Wanderloots in their Hermes Bot Mode tutorial and the companion quick reference notes:

- Hermes Bot Mode quick-reference (Setup Notes, Tools, Skills & Prompts), Wanderloots on Patreon.
- Full Bot Mode video: youtu.be/3RoK0rrOHCA
- Trusted human-in-the-loop LLM wiki review workflow, Wanderloots on Patreon.

Adapt with attribution if you reuse this guide. The example job descriptions and prompts above are adapted from that material as a starting point; tune them to your own needs.

© Billy Forrest @ https://training.forrest.one